

Trilha 2 — Persistência de Dados e Operações CRUD

(Sistema Web de Anúncios Marketplace para ser desenvolvido em Flask)

Atividade referente à implementação completa dos CRUDs (Create, Read, Update, Delete) para todas as entidades definidas no Modelo Entidade-Relacionamento (MER) e na estrutura de navegação do sistema, com persistência em banco de dados via SQLAlchemy.

Repositório GitHub: [<https://github.com/trabalhotrilha3td-sudo/atividade2>]

1. Introdução

Esta atividade dá continuidade ao sistema iniciado na Trilha 1, um sistema web de anúncios (marketplace) construído com o microframework Flask e persistência de dados por meio do SQLAlchemy (ORM) sobre um banco de dados SQLite. O objetivo foi complementar o sistema com os CRUDs completos de todas as entidades planejadas no Modelo Entidade-Relacionamento (MER) e na estrutura de navegação definida anteriormente, permitindo cadastrar, listar, editar e excluir registros de cada entidade diretamente pela interface web.

1.1 Modelo Entidade-Relacionamento (MER)

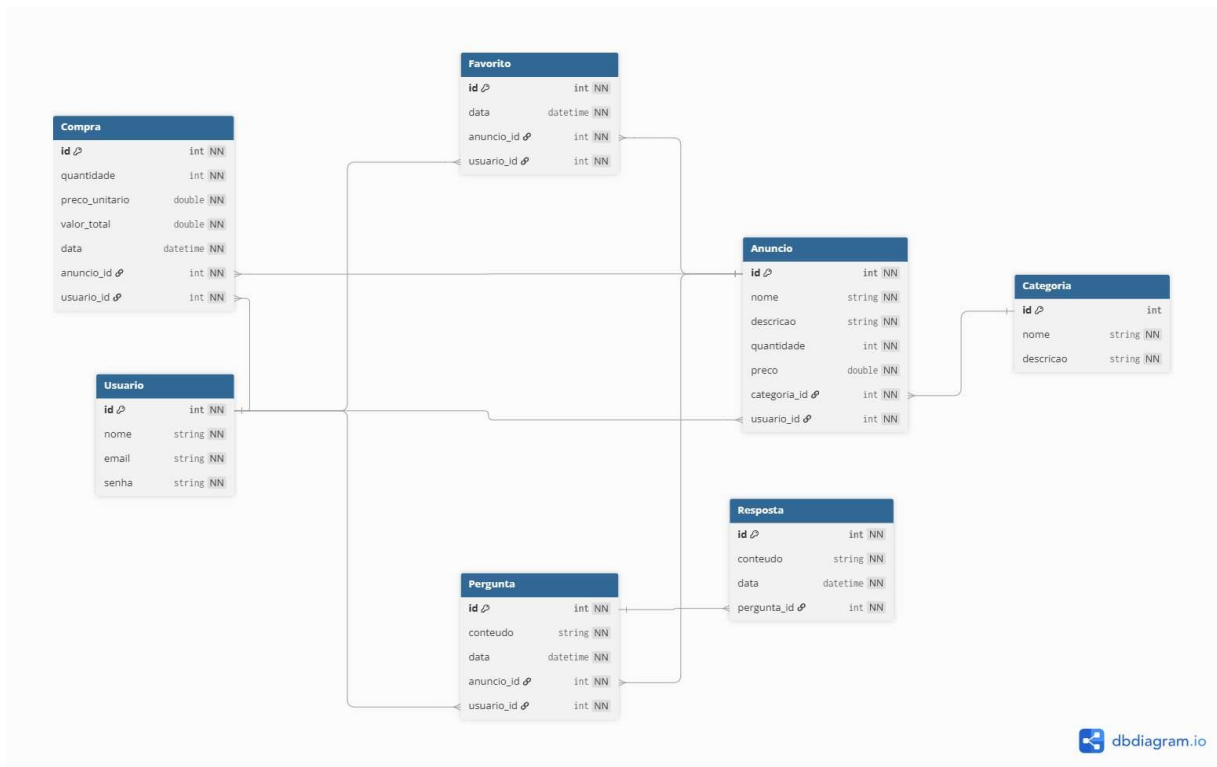


Figura 1 — Diagrama MER do sistema (dbdiagram.io)

1.2 Estrutura de Navegação do Sistema

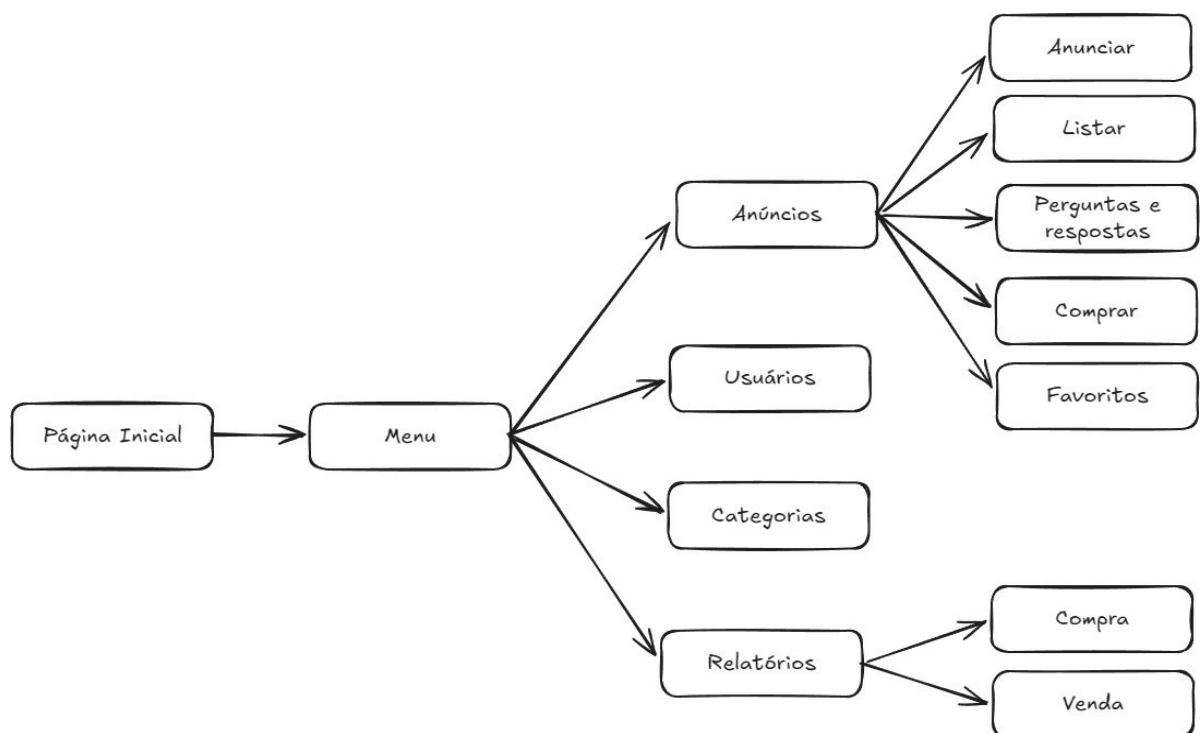


Figura 2 — Fluxo de navegação entre as páginas do sistema

2. CRUDs Implementados por Entidade

Foram implementadas as quatro operações (Create, Read, Update, Delete) para todas as sete entidades do MER: Usuário, Categoria, Anúncio, Pergunta, Resposta, Compra e Favorito. A tabela a seguir resume as rotas Flask responsáveis por cada operação.

Entidade	Create	Read (listar)	Update	Delete
Usuário	POST /usuario/criar	GET /cad/usuario	GET/POST /usuario/editar/<id>	GET /usuario/deletar/<id>
Categoria	POST /categoria/criar	GET /config/categoria	GET/POST /categoria/editar/<id>	GET /categoria/deletar/<id>
Anúncio	POST /anuncio/criar	GET /anuncios/listar	GET/POST /anuncio/editar/<id>	GET /anuncio/deletar/<id>
Pergunta	POST /pergunta/criar	GET /anuncios/pergunta	GET/POST /pergunta/editar/<id>	GET /pergunta/deletar/<id>
Resposta	POST /resposta/criar	GET /anuncios/pergunta	GET/POST /resposta/editar/<id>	GET /resposta/deletar/<id>
Compra	POST /compra/criar	GET /anuncios/compra	GET/POST /compra/editar/<id>	GET /compra/deletar/<id>
Favorito	POST /favorito/criar	GET /anuncios/favoritos	GET/POST /favorito/editar/<id>	GET /favorito/deletar/<id>

Observação: Pergunta e Resposta compartilham a mesma página de listagem (*/anuncios/pergunta*), pois as respostas são exibidas agrupadas junto às perguntas do anúncio, refletindo o relacionamento 1:N entre as duas entidades.

3. Funcionalidades da Interface

Para cada entidade, o sistema oferece:

- Listagem de registros: cada entidade possui uma página própria que consulta o banco via `db.session.scalars(db.select(Modelo)).all()` e exibe os dados em tabela HTML;
- Cadastro de novos registros: formulários HTML enviam dados via método POST para rotas de criação, que instanciam o modelo SQLAlchemy correspondente, adicionam o objeto à sessão e persistem com `db.session.commit()`;
- Edição de registros existentes: rotas que aceitam GET (para exibir o formulário pré-preenchido) e POST (para salvar as alterações), localizando o registro por chave primária com `db.get_or_404()`;
- Exclusão com confirmação: cada link de exclusão dispara uma caixa de confirmação JavaScript (`onclick="return confirm('Deseja deletar?');"`) antes de acionar a rota de exclusão, evitando remoções acidentais.

3.1 Exemplo de rota completa (entidade Categoria)

```
@app.route("/config/categoria")
def categoria():
    categorias = db.session.scalars(db.select(Categoria)).all()
    return render_template('categoria.html', categorias=categorias)

@app.route("/categoria/criar", methods=['POST'])
def criar_categoria():
    nova_cat = Categoria(nome=request.form.get('nome'),
    descricao=request.form.get('descricao'))
    db.session.add(nova_cat)
    db.session.commit()
    return redirect(url_for('categoria'))

@app.route("/categoria/editar/<int:id>", methods=['GET', 'POST'])
```

```
def editar_categoria(id):
    cat_obj = db.get_or_404(Categoria, id)
    if request.method == 'POST':
        cat_obj.nome = request.form.get('nome')
        cat_obj.descricao = request.form.get('descricao')
        db.session.commit()
    return redirect(url_for('categoria'))
    return render_template('editar_categoria.html', categoria=cat_obj)

@app.route("/categoria/deletar/<int:id>")
def deletar_categoria(id):
    cat_obj = db.get_or_404(Categoria, id)
    db.session.delete(cat_obj)
    db.session.commit()
    return redirect(url_for('categoria'))
```

4. Argumentos com Base nos Conceitos das Aulas

Persistência via ORM (SQLAlchemy)

Optou-se por mapear cada entidade do MER como uma classe SQLAlchemy (`db.Model`), conforme trabalhado nas aulas sobre integração entre a aplicação e o banco de dados. Isso evita a escrita manual de SQL, reduz erros e mantém a estrutura das tabelas sincronizada com o modelo de dados por meio de `db.create_all()`.

Relacionamentos e integridade referencial

As chaves estrangeiras (`categoria_id`, `usuario_id`, `anuncio_id`, `pergunta_id`) foram declaradas com `db.ForeignKey`, e os relacionamentos (`db.relationship`) usam `cascade="all, delete-orphan"`. Isso garante que, por exemplo, ao excluir um Anúncio, suas Perguntas, Compras e Favoritos associados sejam removidos automaticamente, mantendo a consistência do banco — um princípio de integridade referencial discutido na trilha.

Padrão Post/Redirect/Get

As rotas de criação e edição redirecionam (`redirect(url_for(...))`) após o `commit()`, em vez de renderizar diretamente uma página de resposta ao POST. Esse padrão evita o reenvio duplicado de formulários ao atualizar a página e reflete a boa prática de separar a operação de escrita da operação de leitura.

Confirmação antes da exclusão

Como o método DELETE não seria acionado por um simples link em HTML puro, optou-se por manter as exclusões como requisições GET protegidas por uma confirmação via JavaScript (`confirm()`) no próprio link. Isso atende ao critério de "exclusão com confirmação" sem exigir formulários adicionais, mantendo a interface simples.

Reutilização de templates com Jinja2

As páginas de listagem e edição reutilizam a mesma estrutura de tabelas e formulários com o motor de templates Jinja2, recebendo os dados do banco como variáveis de contexto (`render_template`). Isso reduz duplicação de HTML e mantém a camada de apresentação desacoplada da lógica de acesso a dados, aplicando a separação de responsabilidades trabalhada em aula.

Regras de negócio na camada de rota

Na criação e edição de Compras, o preço unitário e o valor total são recalculados a partir do preço atual do Anúncio, e a quantidade em estoque do Anúncio é decrementada quando há saldo suficiente. Essa lógica foi mantida na camada de rotas (`controller`), que é responsável por orquestrar as operações entre diferentes entidades antes de persistir os dados.

5. Publicação no GitHub

O código-fonte atualizado, incluindo os CRUDs completos de todas as entidades, os templates HTML e o script de modelos, foi publicado no mesmo repositório público utilizado desde a Trilha 1, com o histórico de commits atualizado até a data de entrega.

Link do repositório: <https://github.com/trabalhotrilha3td-sudo/atividade2>

6. Considerações Finais

Com a implementação apresentada, o sistema passou a oferecer operações completas de cadastro, leitura, edição e exclusão para todas as sete entidades previstas no MER (Usuário, Categoria, Anúncio, Pergunta, Resposta, Compra e Favorito), atendendo aos critérios de avaliação propostos: cobertura total do MER, funcionalidade das operações de CRUD, exclusão com confirmação e publicação do código em repositório público no GitHub.